

Es hora de que pongas en práctica todo lo aprendido. 

En este ejercicio realizaremos autenticación y autorización con Spring Security. Recomendamos crear previamente dos usuarios en la base de datos y colocarles los roles ADMIN y USER de antemano.

Más adelante conseguirás las resoluciones para que valides tus respuestas y puedas monitorear tu progreso. 

¡Manos a la obra!

1. Desafío

- **Desarrolla una aplicación web** que permita a los usuarios:
 - Iniciar sesión.
 - Acceder a dos páginas diferentes: una página llamada `'home.html'` y otra página llamada `'admin.html'`.
 - Los usuarios que tengan el rol de `'ROLE_ADMIN'` podrán acceder tanto a la página `'home'` como a la página `'admin'`.
 - Los usuarios con el rol de `'ROLE_USER'` solo podrán acceder a la página `'home'`.
 - Se implementa un botón de `'Logout'` para permitir que los usuarios cierren su sesión en la aplicación.
 - Al acceder a la página `'admin'`, muestra un mensaje de bienvenida personalizado que **indique el nombre del usuario** autenticado en el controlador.
- Utiliza **Spring Security** para gestionar el control de acceso y asegúrate de que los usuarios deban iniciar sesión antes de acceder a las páginas protegidas.

2. ¿Dónde se lleva a cabo?

Eclipse IDE o Spring Tool Suite

3. Tiempo de dedicación

2 horas

4. Condición

Esta práctica o ejercitación **no requiere ser entregada y/o evaluada** por el mentor. No obstante puedes compartir tus resultados con el resto de los bootcampers y construir conocimiento en conjunto.

Resolución del desafío

Veamos una posible resolución, porque como ya sabemos, en programación es posible resolver la misma problemática de diversas maneras.

Para esta resolución, vamos a construir el proyecto en primer lugar. Puede realizarse también en algún proyecto que vengas practicando y utilices su clase [User/Usuario](#).

En caso de que no tengas un proyecto comenzado, debes tener en cuenta las siguientes dependencias para su construcción (por lo menos):

- Spring Security
- Spring Web
- Spring Data JPA
- MySQL Connector

Clases y vistas:

- Clase Usuario entidad (con atributos nombre, password y rol)
- Interface Usuario repositorio
- Clase Usuario servicio
- Enum Rol {USER, ADMIN}
- Clase de configuración de seguridad
- Vista Home.html
- Vista Admin.html
- Clase Usuario controlador

Bajo esta estructura, la clase de configuración de esta manera:

```

@Configuration
@EnableWebSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter {

    @Autowired
    private UserDetailsService userDetailsService;

    @Autowired
    public void configurerGlobal(AuthenticationManagerBuilder build) throws Exception{
        build.userDetailsService(userDetailsService);
    }

    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http
            .authorizeRequests(requests -> requests
                .antMatchers("/home").hasAnyRole("USER", "ADMIN")
                .antMatchers("/admin").hasRole("ADMIN"))
            .formLogin(login -> login
                .loginPage("/login")
                .permitAll())
            .logout(logout -> logout
                .logoutUrl("/logout")
                .permitAll());
    }
}

```

En este ejemplo para el controlador de usuario, vamos a utilizar `SecurityContextHolder.getContext().getAuthentication()` para obtener el objeto **Authentication** del contexto de seguridad de **Spring Security**, el cual contiene la información del usuario autenticado. Luego, utilizamos `authentication.getName()` para obtener el nombre del usuario autenticado y lo agregamos al modelo como un atributo llamado "nombre":

```

@Controller
public class UsuarioControlador {

    @GetMapping("/home")
    public String home() {
        return "home";
    }

    @GetMapping("/admin")
    public String admin(Model model) {
        // Obtiene el objeto Authentication desde el contexto de seguridad de Spring Security
        Authentication authentication = SecurityContextHolder.getContext().getAuthentication();

        // Obtiene el nombre del usuario autenticado
        String nombre = authentication.getName();

        // Agrega el nombre del usuario al modelo
        model.addAttribute("nombre", nombre);

        return "admin.html";
    }
}

```

Finalmente vamos a configurar las vistas html (o JSP, ¡si prefieres!) de la siguiente manera:

```
say,
<!-- login.html -->
<form action="/login" method="post">

    <input type="text" id="nombre" name="nombre" required>

    <input type="password" id="password" name="password" required>

    <button type="submit">Login</button>
</form>

<!-- home.html -->
<h1>HOME (acceso User y Admin!)</h1>

<a href="/logout">Logout</a>

<!-- admin.html -->
<h1>Bienvenido, ${nombre}, a la página para Admin!</h1>

<a href="/logout">Logout</a>
```